phylo

M. J. Wilson

September 2, 2026

Abstract

`phylo` searches the space of phylogenetic tree topologies with reinforcement learning, scoring candidates by their likelihood under a model of character substitution. It is built by an agent working to a fixed contract: every claim is pinned by a test against an independent oracle, every figure is regenerated from the code it reports on, and the conventions that make that reviewable are stated in the repository rather than rediscovered per change. The optimization machinery is model-agnostic by construction — an unconstrained parameter vector, a differentiable objective, and a map back to named constrained parameters — so the same optimizer fits a Potts chain, a hidden Markov model and a phylogeny, and discrete moves sit outside it as an operation that builds a new objective rather than a step within one. On the phylogenetic side that currently delivers a validated simulator, Felsenstein pruning across four backends, and branch lengths, exchangeabilities and stationary frequencies fitted by automatic differentiation and recovered within their confidence intervals. This document states the theory behind all of it, and holds the captions for the figures the quality-assurance suite emits, so that a plot and its description cannot drift apart.

Contents

Intended reader. Intended reader is: well-educated developer with scientific and performance-computing background, but not an expert in a given application, e.g. phylogenetics. The body stays streamlined — citing sources rather than re-deriving standard material inline — and pushes required appliation background into Appendix A, referenced from the point of use. It is a higher-level overview of the current best-known approach (simulation, models) in line with the roadmap, not an exhaustive record; detail, plots, and results for support are linked to dedicated supporting documents, e.g. the results of a dedicated study to find the best of a number of approaches.

1 Scope and status

The goal is Milestone 8 of the project roadmap: a policy that proposes tree rearrangements and beats classical hill-climbing on held-out simulated data at a matched evaluation budget. Everything below is either what that needs or what has been built towards it.

Built and validated: the k-state Jukes–Cantor and general time-reversible substitution models with their simulators; Felsenstein pruning in vectorized NumPy, differentiable PyTorch and Rust, each pinned against brute-force marginalization; the model-agnostic optimization interface and its optimizer, with confidence intervals from the observed information; branch lengths, exchangeabilities and stationary frequencies fitted and recovered against known truth; and NNI and SPR neighbourhood generators checked exhaustively against closed-form neighbour counts.

Not built: subtree and site compression, a canonical topology key, GPU dispatch, the search loop that joins the move sets to the optimizer, the benchmark harness, and the reinforcement-learning agent itself. Sections covering those describe intent, and say so; everything else describes either mathematics that does not depend on this implementation, or behaviour a test pins.

2 Notation

Two vocabularies are kept apart, because the machinery in this document is deliberately indifferent to which application it serves. The first describes an optimization problem; the second describes a phylogeny. Nothing in the first refers to a tree.

2.1 The optimization abstraction

An objective is a differentiable scalar $\mathcal{L}(\boldsymbol{\theta})$ over an unconstrained vector $\boldsymbol{\theta} \in \mathbb{R}^d$, together with a map from $\boldsymbol{\theta}$ to the constrained parameters a model is actually stated in — positive rates, distributions on a simplex. Minimization is the convention throughout, so a likelihood-based objective is a negative log-likelihood.

A *discrete move* changes the structure being fitted, and therefore changes both the meaning and the dimension d of $\boldsymbol{\theta}$. It is not a step within an optimization; it constructs a new objective.

Symbol	Meaning
$\boldsymbol{\theta}$, d	unconstrained parameter vector and its dimension
$\mathcal{L}(\boldsymbol{\theta})$	scalar objective, minimized
$\nabla\mathcal{L}$, $\nabla^2\mathcal{L}$	its gradient and Hessian

Table 1: Symbols for the optimization abstraction, which is shared by every application.

2.2 The phylogenetic application

Let $\mathcal{A}$ be the finite state space of a single character, with $k = |\mathcal{A}|$; for nucleotide data $\mathcal{A} = \{A, C, G, T\}$ and $k = 4$. The data $\mathbf{D}$ are an alignment of n taxa over m sites, so $\mathbf{D} \in \mathcal{A}^{n \times m}$, with $\mathbf{D}_{\cdot s}$ the observed column at site s.

A phylogeny is a pair $(\mathcal{T}, \boldsymbol{\tau})$: a topology $\mathcal{T}$ whose n leaves carry the taxa, and a vector $\boldsymbol{\tau}$ of non-negative branch lengths, measured in expected substitutions per site. Internal nodes represent unobserved ancestors, and ρ denotes the root.

Symbol	Meaning
$\mathcal{A}$, k	character state space and its size
n, m	number of taxa, number of sites
$\mathcal{T}$	tree topology
$\boldsymbol{\tau}$	vector of branch lengths
ρ	the root node
$\mathbf{Q}$	instantaneous rate matrix, $k \times k$
$\mathbf{P}(t)$	transition-probability matrix over a branch of length t
$\boldsymbol{\pi}$	root (and, under reversibility, stationary) distribution
$L_u(i)$	partial likelihood at node u given state i

Table 2: Symbols for the phylogenetic application. The substitution model these describe is developed in Appendix B.

3 The likelihood

Sites are modelled as independent given $(\mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}, \boldsymbol{\pi})$, so the log-likelihood is a sum over columns,

$$\log \mathcal{L}(\mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}, \boldsymbol{\pi} \mid \mathbf{D}) = \sum_{s=1}^{m} \log \Pr(\mathbf{D}_{\cdot s} \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}, \boldsymbol{\pi}). \tag{1}$$

The site term marginalizes over the unobserved states of every internal node. Written directly, that is a sum over k^{n-1} assignments — intractable beyond a handful of taxa. Felsenstein's pruning algorithm computes it in time linear in n by exploiting the tree's conditional independence structure [8]; it is the sum-product algorithm on a tree [14, 12].

3.1 Pruning

Define the partial likelihood $L_u(i)$ as the probability of the data at the leaves below node u, given that u is in state i. At a leaf with observed state a, $L_u(i) = \mathbb{1}[i = a]$. At an internal node u with children v joined by branches of length t_v,

$$L_u(i) = \prod_{v \in \text{children}(u)} \left(\sum_{j \in \mathcal{A}} P_{ij}(t_v)\, L_v(j) \right), \tag{2}$$

and at the root r the site likelihood is

$$\Pr(\mathbf{D}_{\cdot s} \mid \cdot) = \sum_{i \in \mathcal{A}} \pi_i\, L_r(i). \tag{3}$$

Algorithm 1 Felsenstein pruning for one site

Require: topology $\mathcal{T}$, branch lengths $\boldsymbol{\tau}$, rate matrix $\mathbf{Q}$, root distribution $\boldsymbol{\pi}$, observed column $\mathbf{D}_{\cdot s}$
Ensure: site likelihood $\Pr(\mathbf{D}_{\cdot s} \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}, \boldsymbol{\pi})$
1: **for all** nodes u in post-order **do**
2: **if** u is a leaf with observed state a **then**
3: $L_u(i) \leftarrow \mathbb{1}[i = a]$ for all $i \in \mathcal{A}$
4: **else**
5: $L_u(i) \leftarrow \prod_{v \in \text{children}(u)} \sum_j P_{ij}(t_v) L_v(j)$ for all $i \in \mathcal{A}$
6: **end if**
7: **end for**
8: **return** $\sum_i \pi_i L_r(i)$

Algorithm 1 costs $O(nk^2)$ per site against $O(k^{n-1})$ for direct marginalization. The two agree exactly, which makes brute-force marginalization on small trees the reference implementation the pruning code is tested against.

Partial likelihoods underflow for realistic m and n, so the implementation will rescale them per node and accumulate the log of the scaling factors. Rescaling changes the arithmetic but not the result: the rescaled and direct computations must agree to within floating-point tolerance on small problems where both run.

3.2 Differentiable backends

Branch lengths $\boldsymbol{\tau}$ are what gradient-based fitting and tree search differentiate through, so an accelerated backend must keep them in the autodiff graph rather than baking them into topology state. The PyTorch backend (`pruning_torch.py`, float64, CPU) takes $\boldsymbol{\tau}$ as a tensor separate from the topology and is accepted when its log-likelihood agrees with the NumPy reference to atol $= 10^{-9}$ – the same bound as the brute-force check above – never relaxed to accommodate a discrepancy. Gradients obtained from `torch.autograd` are checked against central finite differences of the NumPy oracle (step $\epsilon = 10^{-6}$) to atol $= 10^{-6}$, the coarser bound set by the finite-difference truncation and rounding error at that step size, not by the autodiff computation itself.

3.3 Rate variation across sites

Sites evolve at different rates. The standard treatment draws a site-specific rate multiplier r from a discretized gamma distribution with mean one, and averages the site likelihood over the

c rate categories,

$$\Pr(\mathbf{D}_{\cdot s} \mid \cdot) = \frac{1}{c} \sum_{g=1}^{c} \Pr(\mathbf{D}_{\cdot s} \mid \mathcal{T}, r_g \boldsymbol{\tau}, \mathbf{Q}, \boldsymbol{\pi}), \tag{4}$$

multiplying the cost of a likelihood evaluation by c [8]. *Not yet implemented.*

4 Simulation

Simulation is how the implementation is tested: draw data from known parameters, then check that each component recovers what the generating model implies. Generation runs in pre-order, the reverse of pruning.

Algorithm 2 Simulating an alignment under $(\mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}, \boldsymbol{\pi})$

Require: topology $\mathcal{T}$, branch lengths $\boldsymbol{\tau}$, rate matrix $\mathbf{Q}$, root distribution $\boldsymbol{\pi}$, sites m, seeded generator
Ensure: alignment $\mathbf{D} \in \mathcal{A}^{n \times m}$

```
for s ← 1 to m do
    draw the root state x_r ~ π
    for all nodes u in pre-order, u ≠ r with parent p and branch t_u do
        draw x_u ~ P(t_u)_{x_p,·}
    end for
    record x_u at every leaf u as column s
end for
return D
```

Every draw uses an explicitly seeded generator, so a simulated dataset is reproducible from its seed. Topology, branch lengths, k, $\boldsymbol{\pi}$, seed and site count are declared in a parameters file, and the generating truth is retained alongside the alignment, so a dataset always carries what produced it. Simulated substitution frequencies are checked against (15) within a declared Monte Carlo tolerance.

Table 4 in Section 9 lists every simulation regression fixture at the problem sizes actually run, read directly from each fixture's yaml; Figure 2 there shows a complete generated instance at the smallest of them — the Newick string and the leading simulated sites of the alignment.

5 Tree search

5.1 The size of the problem

The number of distinct unrooted binary topologies on n taxa is the double factorial $(2n-5)!!$ [24], which grows faster than exponentially (Table 3). Exhaustive search is impossible past a handful of taxa, and finding the optimal tree under either the parsimony or the likelihood criterion is NP-hard [8]. Practical inference is therefore heuristic: start somewhere, propose rearrangements, keep what scores better.

Taxa n	Unrooted binary topologies $(2n-5)!!$
10	2.0×10^{6}
20	2.2×10^{20}
50	2.8×10^{74}

Table 3: Growth of tree space with the number of taxa.

5.2 Move sets

Two rearrangement neighbourhoods define the search graph [8]: NNI and SPR, defined in Appendix A. NNI's neighbourhood is small, cheap, and local; SPR's is quadratic in n, so each step costs more, but its larger reach escapes local optima that trap NNI.

Since neighbouring topologies differ only near the affected edges, most partial likelihoods are unchanged between a tree and its neighbour. Caching and recomputing only the invalidated path is what makes tree search affordable, and is a design requirement on the likelihood engine rather than an optimization to add later.

5.3 Canonical serialization

Newick notation is not a unique encoding: one topology admits many strings, differing by the order in which children are written and by where the string is rooted. A search that keys on the raw string therefore fails to recognize a tree it has already scored, and a model trained on raw strings learns to distinguish spellings that carry no phylogenetic meaning. Algorithm 3 removes both problems by fixing a representative.

Algorithm 3 Canonical form of an unrooted binary tree with labelled leaves

Require: tree $\mathcal{T}$ with leaves labelled from a totally ordered set
Ensure: a string identifying $\mathcal{T}$'s topology uniquely

root $\mathcal{T}$ at the leaf whose label is smallest in that order
for all nodes u in post-order **do**
    if u is a leaf **then**
        $\kappa(u) \leftarrow \text{label}(u)$
    else
        sort the children v of u by $\kappa(v)$
        $\kappa(u) \leftarrow$ the concatenation of $\kappa(v)$ in that order
    end if
end for
return κ(root), emitting children in sorted order

Rooting at the smallest label is well defined for any labelled tree, and the child order is fixed bottom-up by keys that depend only on the subtree below, so the output depends on the topology alone. Hashing the keys keeps the cost at $O(n \log n)$ [6]. Branch lengths are carried separately, as a vector in canonical node order, so one topology under two length assignments shares a topology key — which is what makes the key usable for memoizing likelihood evaluations across a search.

Two properties are required of any implementation, and both are tested rather than assumed: two trees produce the same string exactly when they are the same topology, checked by randomly re-rooting and permuting the children of one tree and by comparing against distinct trees; and parsing a canonical string reproduces the tree that produced it.

5.4 Extensions to the move set

Three extensions target the same bottleneck: the number of likelihood evaluations a search spends. All three are proposals. *Not yet implemented.*

Bounding whole sets of trees. Scoring every neighbour exactly wastes work when most are poor. A cheap *upper* bound on the likelihood attainable anywhere within a set of topologies lets the search discard the whole set, in the manner of branch and bound [6]: when the bound falls below the best log-likelihood already achieved, no member of the set can win. Two sources of

bounds are worth trying: relaxations of the pruning recursion (2), which replace the product over children by a quantity that is cheaper and provably no smaller; and compressed summaries of the alignment, where identical site patterns already collapse and coarser encodings trade tightness for speed [2]. Soundness is the whole game — a bound that can fall below an attainable likelihood silently discards the optimum — so each candidate bound needs a proof, and a test that searches small trees exhaustively for a counterexample.

Multi-moves. Composing several NNI or SPR rearrangements into one action reaches topologies no single move reaches, at the cost of a much larger action space; in reinforcement-learning terms these are temporally extended actions [25]. Rather than fixing how many rearrangements a compound move contains, draw the composition during training from a Dirichlet process, so the number of distinct compound moves the agent maintains grows with the evidence for them instead of being set in advance [14]. The measure of success is unchanged: fewer likelihood evaluations for the same likelihood reached.

A transformer policy over Newick strings. A topology serializes to a Newick string, which a transformer can consume directly — tokenize the string, pretrain on simulated trees, then fine-tune the policy with the usual policy-gradient machinery [23, 13, 25]. The input is the canonical form of Section 5.3, which settles the encoding question: one topology, one string. The alternative, an architecture invariant to child order and root placement, buys the same property at greater cost and without supplying the memoization key the search needs anyway. Compression then applies on top, replacing recurring subtrees by dictionary symbols — an extended alphabet over the canonical string [2] — which shortens the token sequence a transformer must attend over.

5.5 Parsimony and likelihood

The *large parsimony* problem asks for the topology minimizing the number of state changes needed to explain the data. We keep the search problem and replace the criterion: candidates are scored by the maximized log-likelihood of (1), with the continuous parameters $(\boldsymbol{\tau}, \mathbf{Q}, \boldsymbol{\pi})$ fitted per topology. Likelihood is the more principled criterion — it models the substitution process rather than counting changes, and it is statistically consistent where parsimony can be positively misleading [8] — at the cost of a far more expensive score.

6 Gradients

Fitting $(\boldsymbol{\tau}, \mathbf{Q}, \boldsymbol{\pi})$ for a candidate topology is a continuous optimization, and gradients make it tractable. Differentiating (11) with respect to a branch length gives

$$\frac{\partial \mathbf{P}(t)}{\partial t} = \mathbf{Q}\exp(\mathbf{Q}t) = \mathbf{Q}\,\mathbf{P}(t). \tag{5}$$

For a reversible $\mathbf{Q}$ with eigendecomposition $\mathbf{Q} = \mathbf{U}\Lambda\mathbf{U}^{-1}$, both the exponential and its derivative are diagonal in the eigenbasis,

$$\mathbf{P}(t) = \mathbf{U}e^{\Lambda t}\mathbf{U}^{-1}, \qquad \frac{\partial \mathbf{P}(t)}{\partial t} = \mathbf{U}\Lambda e^{\Lambda t}\mathbf{U}^{-1}, \tag{6}$$

so a single decomposition serves every branch. Gradients of the log-likelihood with respect to all branch lengths follow from reverse-mode automatic differentiation through the pruning recursion (2), at a cost proportional to one forward evaluation [10, 22]. The optimizer consuming them is a standard constrained quasi-Newton or trust-region method [19]; the parameters carry constraints

— non-negative branch lengths, a normalized $\boldsymbol{\pi}$ — so the geometry of the statistical manifold, and the natural gradient it induces, is the principled setting [1].

Analytic and automatic gradients are checked against central finite differences,

$$\frac{\partial f}{\partial \theta} \approx \frac{f(\theta + h) - f(\theta - h)}{2h}, \tag{7}$$

with h chosen to balance truncation against round-off. This check is a required test for any differentiable component, and is implemented for every objective the optimizer accepts.

6.1 The optimization interface

The optimizer is deliberately ignorant of what it optimizes. It takes an unconstrained parameter vector, a differentiable scalar to minimize, and a map back to the named constrained parameters; nothing in it refers to a tree, and a test asserts that the module imports nothing from the simulation, likelihood or search packages. Constraints are imposed by construction rather than by projection — positive scales through a log or softplus map, a distribution through a softmax — so no step can leave the feasible set.

One detail of that map is load-bearing rather than cosmetic. A softmax over n logits is invariant to adding a constant to all of them, so one direction of the parameter space is exactly flat; the observed information is then singular and the intervals below are undefined. Pinning the first logit removes that direction and makes the map a bijection onto the simplex. The same issue appears as a physical gauge in the Potts field of Section 9 and is fixed the same way.

Discrete moves sit outside this interface. A move changes the structure — a different topology, chain length or state count — and so changes what the parameter vector means and how long it is. It therefore constructs a *new* objective rather than stepping inside a fit over a fixed-length vector, and the loop that proposes moves owns that construction. This is what lets the tree search of Section 5 and the continuous fit share an optimizer without either knowing about the other.

Fitted parameters are reported with intervals derived from the observed Fisher information — the Hessian of the negative log-likelihood at the optimum — pushed through the constraint map by the delta method. Those intervals are asymptotic, and Section 9 reports how fast they converge on their nominal coverage rather than assuming they have.

That machinery also decides which parameters may be reported at all. An observed information with a flat direction is singular, and the interval it implies is not merely wide but undefined; the check is on the ratio of its smallest to its largest eigenvalue, since rounding leaves an exactly flat direction merely tiny rather than zero. For the phylogenetic model this is not a hypothetical: the two branches below a rooted binary tree's root realize a ratio of 6.7×10^{-8}, against 6.1×10^{-2} for the same tree with that pair fitted as their sum. Section 9 shows the underlying invariance directly.

7 Reinforcement-learning formulation

Classical heuristics propose moves by a fixed, hand-designed rule. We ask whether a learned policy proposes better ones, and cast the search as a Markov decision process $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ [25]:

State. The current topology with its fitted continuous parameters, together with a summary of the alignment.

Action. A move drawn from the NNI or SPR neighbourhood of the current topology.

Transition. Applying the move and refitting the continuous parameters — deterministic given the action, up to the optimizer.

Reward. The improvement in maximized log-likelihood, $R = \log \mathcal{L}(\mathcal{T}') - \log \mathcal{L}(\mathcal{T})$.

Episode. One search trajectory from a starting tree, terminating on a step budget or when no move improves the score.

The natural performance measure is not the likelihood reached but the likelihood reached *per likelihood evaluation*: every method converges given unbounded evaluations, so the comparison against a hill-climbing baseline has to be made at equal budget.

7.1 Why this reward, and what it implies

The reward is a difference of log-likelihoods, which is not an arbitrary choice: it makes the undiscounted return telescope. Over a trajectory $\mathcal{T}_0, \ldots, \mathcal{T}_T$,

$$\sum_{t=0}^{T-1} R_t = \sum_{t=0}^{T-1} \big[\log \mathcal{L}(\mathcal{T}_{t+1}) - \log \mathcal{L}(\mathcal{T}_t)\big] = \log \mathcal{L}(\mathcal{T}_T) - \log \mathcal{L}(\mathcal{T}_0), \tag{8}$$

so the return of an episode is exactly the total improvement it achieved, independent of the path taken to it. A policy maximizing expected return is therefore maximizing the quantity the benchmark measures, with no reward shaping to justify separately.

This forces $\gamma = 1$. Any $\gamma < 1$ breaks the telescoping and prefers improvement found early over the same improvement found late — defensible when a trajectory may not terminate, but here episodes are capped by an evaluation budget, so the return is a finite sum and needs no discounting to converge.

7.2 Policy gradient and its baseline

The action set is the move neighbourhood, whose size varies with n and with the move type, so the policy scores candidate moves rather than indexing a fixed action space: $\pi_{\boldsymbol{\theta}}(a \mid s) \propto \exp f_{\boldsymbol{\theta}}(s, a)$ over the available a. The gradient of expected return is the standard score-function estimator [25],

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}\Big[\sum_{t=0}^{T-1} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t \mid s_t)\big(G_t - b(s_t)\big)\Big], \qquad G_t = \sum_{u=t}^{T-1} R_u, \tag{9}$$

with $b(s_t)$ a baseline that leaves the estimator unbiased — it multiplies a term of zero expectation — while reducing its variance. The variance matters more here than the bias-free property: G_t is a sum of log-likelihood differences whose scale depends on the alignment, so without a baseline the gradient magnitude varies by orders of magnitude between problems. A learned state-value estimate $\hat{v}(s_t)$ is the usual choice and is the one that makes $G_t - b(s_t)$ an advantage.

7.3 Credit assignment

Tree search makes the credit-assignment problem sharp. A move that unlocks a region of the space may itself lower the likelihood, with the gain arriving several moves later; by (8) the return correctly attributes that gain to the whole trajectory, but the per-step estimator in (9) must still spread it over the actions that earned it. Two consequences for an implementation. Episodes cannot be truncated arbitrarily, since a truncation that lands between the sacrifice and the payoff teaches the opposite of the truth. And a greedy baseline — hill climbing — is exactly a policy that refuses every negative R_t, so beating it requires the agent to accept some, which is the behaviour the variance-reduced estimator has to be able to learn rather than average away [25, 13].

Not yet implemented. Milestone 8 is where this is built; the search loop it will replace, hill climbing over the same move sets, is itself not yet built.

8 Computational structure

Likelihood evaluation dominates the run time: every proposed move costs at least one evaluation, and search proposes many. Its structure is favourable — sites are independent by (1), and the per-node work in (2) is a small dense matrix–vector product — so the computation is a batched, data-parallel reduction over a tree.

The project's performance rule follows from that. Where a kernel plausibly achieves an order of magnitude over the vectorized NumPy reference at the sizes actually run, it targets the GPU through PyTorch, Triton, or JAX, which supply autodiff and batching alongside the parallelism; otherwise the work goes to the Rust backend. The threshold is a measured benchmark, not an expectation, and every accelerated kernel keeps its reference implementation as a pinned oracle. Kernel-level considerations — memory hierarchy, occupancy, parallel reduction — follow Hwu et al. [11]; the CPU-side counterparts, caching and where time actually goes, follow Bryant and O'Hallaron [4], and the backend itself is Rust [3].

Memory is the constraint to watch: partial likelihoods occupy $O(n\,m\,k\,c)$ floating-point values for n taxa, m sites, k states, and c rate categories, before any caching of unchanged subtrees across proposals.

9 Quality assurance

The test suite is expected to establish scientific validity, not to exercise syntax: fixtures come from component-wise simulation under known parameters, expected values come from analytic or brute-force sources, and the invariants stated above — stochasticity of $\mathbf{P}(t)$, detailed balance (12), agreement of pruning with direct marginalization, gradients against finite differences (7), re-rooting invariance — are checked directly.

Part of that suite emits scientific-style figures and tables: parameter recovery against simulated truth, likelihood surfaces, convergence traces, and timing comparisons. **Their captions belong in this section**, added in the same change that adds the figure, so that a plot and its description cannot drift apart.

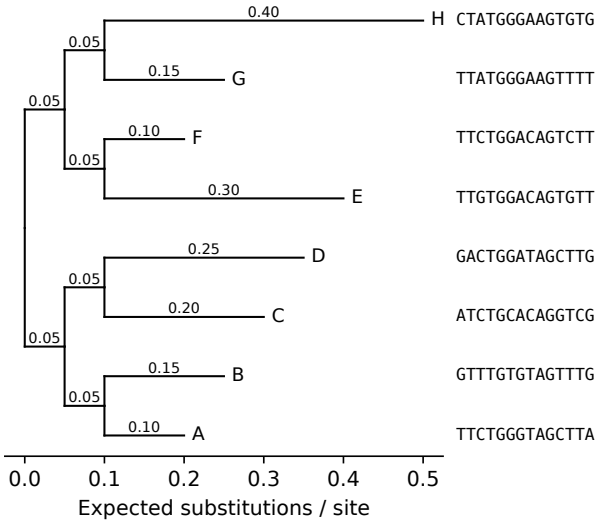

Figure 1: Simulated topology for 8 taxa under the Jukes-Cantor model (seed 20260904), branch lengths labelled in expected substitutions per site. The first 14 simulated sites are shown beside each leaf, in tree order, so the alignment the likelihood consumes can be read off against the topology that generated it.

Every figure in this section is regenerated on every build of this document, from the same regression fixtures the test suite runs, and its caption is read verbatim from the file the generating script writes alongside it. A figure and its description therefore cannot drift apart, and neither can drift from what is actually tested; that guarantee is stated here once and holds for all of them.

Figure 1 is the first: the topology and branch lengths an alignment is drawn over, at the 8-taxon fixture, with the leading simulated sites shown beside each leaf.

Table 4 tabulates the taxa count, site count, seed and Monte Carlo tolerance of every simulation regression fixture, read from the fixtures themselves rather than restated by hand.

Fixture	Taxa	Sites	Seed	Tolerance
`simulation_params.yaml`	4	200_000	20260902	0.01
`simulation_params_small_sites.yaml`	4	20_000	20260903	0.03
`simulation_params_8taxa.yaml`	8	200_000	20260904	0.01

Table 4: Problem-size parameters across the 3 simulation regression fixtures backing the Jukes-Cantor simulation test: taxa count, site count, seed, and the Monte Carlo tolerance each validates simulated substitution frequencies within.

Figure 2 shows a complete generated instance at the smallest of those sizes: the 4-taxon topology with its branch lengths — ρ the root, Greek letters its internal ancestors — and the leading simulated sites of the alignment, nucleotide-coded for this $k = 4$ model.

$$(A\_0.1, B\_0.25, (C\_0.15, D\_0.4)\alpha\_0.05)\rho$$

	0	1	2	3	4	5	6	7	8	9
A	T	G	A	T	G	G	G	T	A	G
B	A	G	T	T	T	T	G	T	A	G
C	T	G	A	T	T	G	G	T	A	G
D	G	G	A	T	C	T	G	C	T	A

Figure 2: Worked example: 4-taxon alignment simulated under the Jukes-Cantor model (seed 20260902, 200_000 sites total), showing the topology with branch lengths – rho the root, Greek letters its internal ancestors – and the first 10 of 200_000 simulated sites.

Figure 3 closes the simulation section by checking the simulator against the model it is supposed to realize. The curves are the closed-form transition probabilities of eq. (15); the markers are the substitution frequencies the simulator actually produced, one pair per branch. Agreement here is what licenses using simulated data as ground truth everywhere else in this document: the simulator is validated against the analytic form, never against the pruning code it feeds.

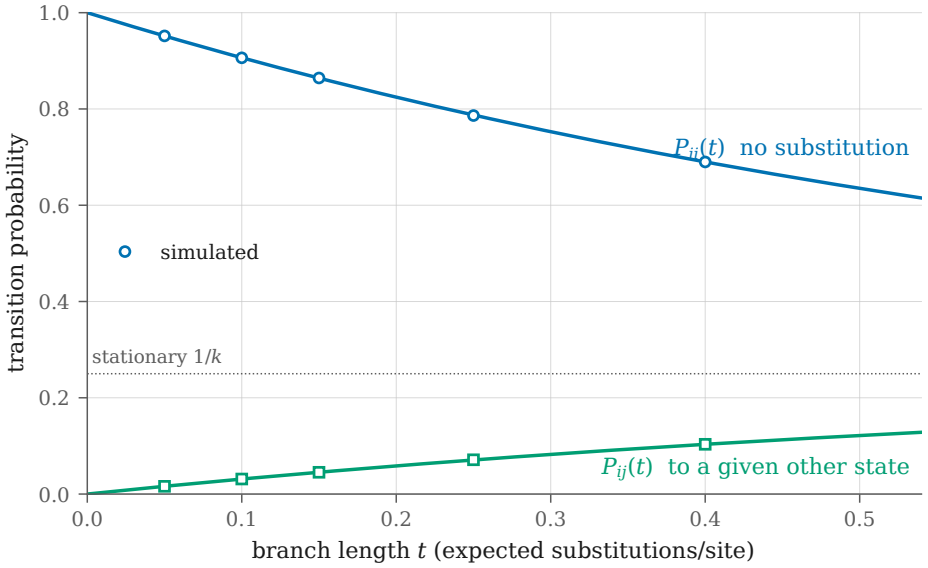

Figure 3: Closed-form Jukes-Cantor transition probabilities for k = 4 (curves) against the frequencies phylo.sim.simulate produced (markers), one pair per branch of the 5-branch fixture tree. Simulated at seed 20260902 over 200000 sites. Largest deviation 1.07e-03, within the fixture's declared Monte Carlo tolerance of 1e-02. The simulator is validated against this analytic form, never against the likelihood code it feeds.

Figure 4 is the acceptance test for the optimizer of Section 6.1, run on two models that share nothing but a factorized structure: a one-dimensional Potts chain in an external field, and a discrete hidden Markov model. Neither is phylogenetic, which is the point — an interface demonstrated on a single model is shaped by that model. Each fitted parameter is plotted against the value that generated the data, with the interval the observed information implies; markers are open where the interval misses, so the figure states its own coverage.

Coverage on one dataset is a draw, not a rate. Figure 5 refits both models over independent datasets at a range of sizes and plots the fraction of intervals containing the truth. Both converge on the nominal rate from opposite sides, which is the behaviour of an asymptotic approximation; a wrong variance formula would not improve with sample size. The hidden Markov model approaches from below for two identifiable reasons absent from the Potts chain: emission probabilities fitted near zero, where a Wald interval on the log scale is a poor approximation, and the post-selection cost of aligning the hidden-state permutation the likelihood is invariant to.

Figure 6 applies that optimizer to the phylogenetic case Milestone 4 names: branch lengths fitted by automatic differentiation through the pruning recursion, validated against the parameters that generated the alignment. Panel (b) is the reason the fit has one fewer parameter than the tree has branches. Moving length between the two branches below a rooted binary tree's root, holding their sum fixed, leaves the log-likelihood unchanged to floating-point precision, while the same move between two non-root siblings costs tens of log units. This is the pulley principle of Section B.1 seen as a statement about estimability: the pair contributes one parameter, not two, and fitting both would leave the observed information singular and every interval in panel (a) undefined.

Figure 7 fits the substitution model itself — the exchangeabilities and stationary frequencies of Section B.2 — alongside the branch lengths. Panel (a) recovers an asymmetric generating truth. Panel (b) is the half that is easy to omit: given data simulated under Jukes–Cantor, the general model must return equal exchangeabilities and a uniform $\boldsymbol{\pi}$ rather than inventing

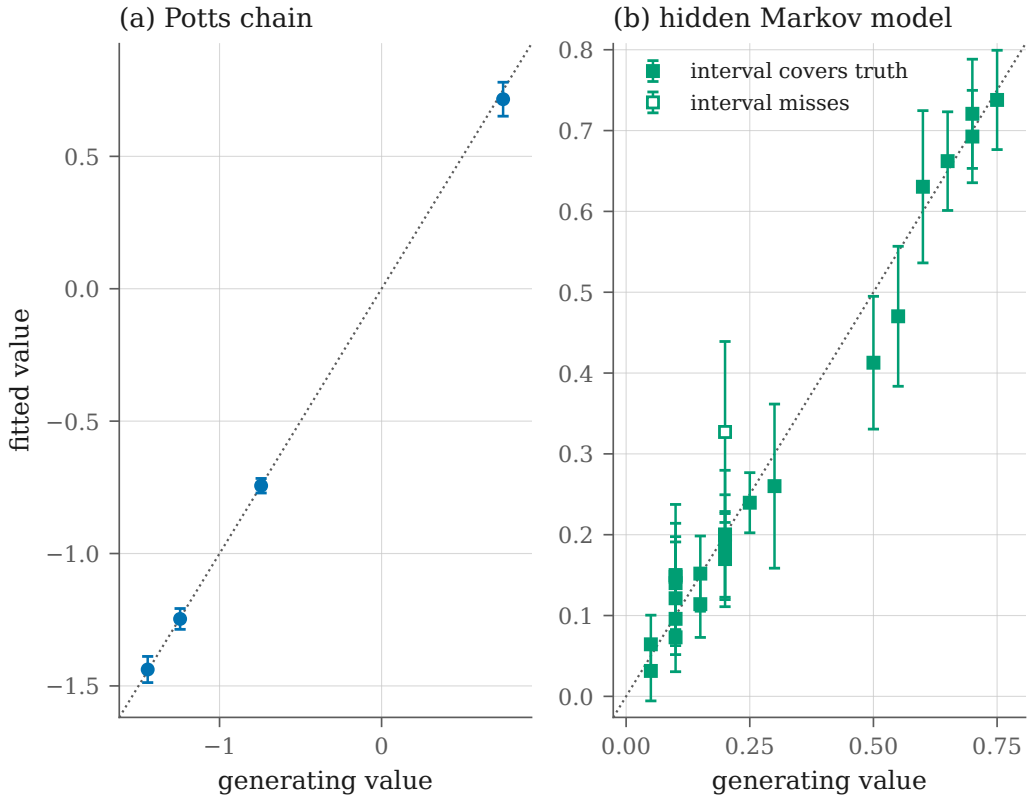

Figure 4: Parameter recovery for the two reference instances of the optimization interface, fitted by the same model-agnostic phylo.opt.fit. Bars are 95 percent Wald intervals from the observed information, propagated to the reported parameters by the delta method; the dotted line is y = x. (a) A 3-state Potts chain, 400 chains of length 12, seed 20260902: coupling and gauge-fixed field, 4 of 4 intervals covering (1.00). (b) A 3-state, 4-symbol hidden Markov model, 600 sequences of length 15, seed 20260903: initial, transition and emission probabilities after aligning the hidden-state permutation the likelihood is invariant to, 23 of 24 intervals covering (0.96). Coverage on a single dataset is a draw, not a rate: the nominal 0.95 is checked over independent replicates by the fitting regression suite, and against sample size in the companion coverage figure.

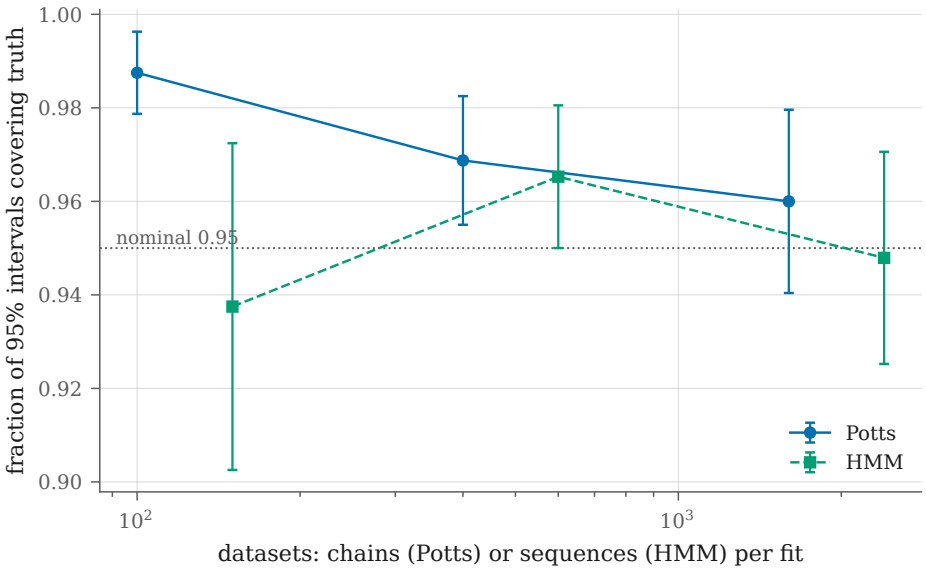

Figure 5: Realized coverage of the 95 percent Wald intervals built from the observed information, against the amount of data each fit sees. Bars are binomial standard errors on the realized fraction. Both instances converge on the nominal rate and neither sits on it at the smallest sample, which is what an asymptotic approximation does and a wrong formula does not. They approach from opposite sides. The Potts chain over-covers and comes down: $158/160 = 0.988$ at 100 chains, $96/100 = 0.960$ at 1600. The hidden Markov model under-covers and comes up: $45/48 = 0.938$ at 150 sequences, $91/96 = 0.948$ at 2400. The under-coverage has two identifiable sources absent from the Potts chain – some emission probabilities are fitted near zero, where a Wald interval on the log scale is a poor approximation, and aligning the hidden-state permutation to the truth is a post-selection step that costs a little coverage. Seeds are fixed, so every point is reproducible rather than redrawn. At the smallest hidden Markov size 6 of 18 fits reached the boundary of the parameter space – an emission probability estimated at zero – where the observed information is singular and there is no interval to report. Those fits are excluded from the counts above and stated here rather than dropped silently, since excluding them unannounced would select for the well-behaved samples.

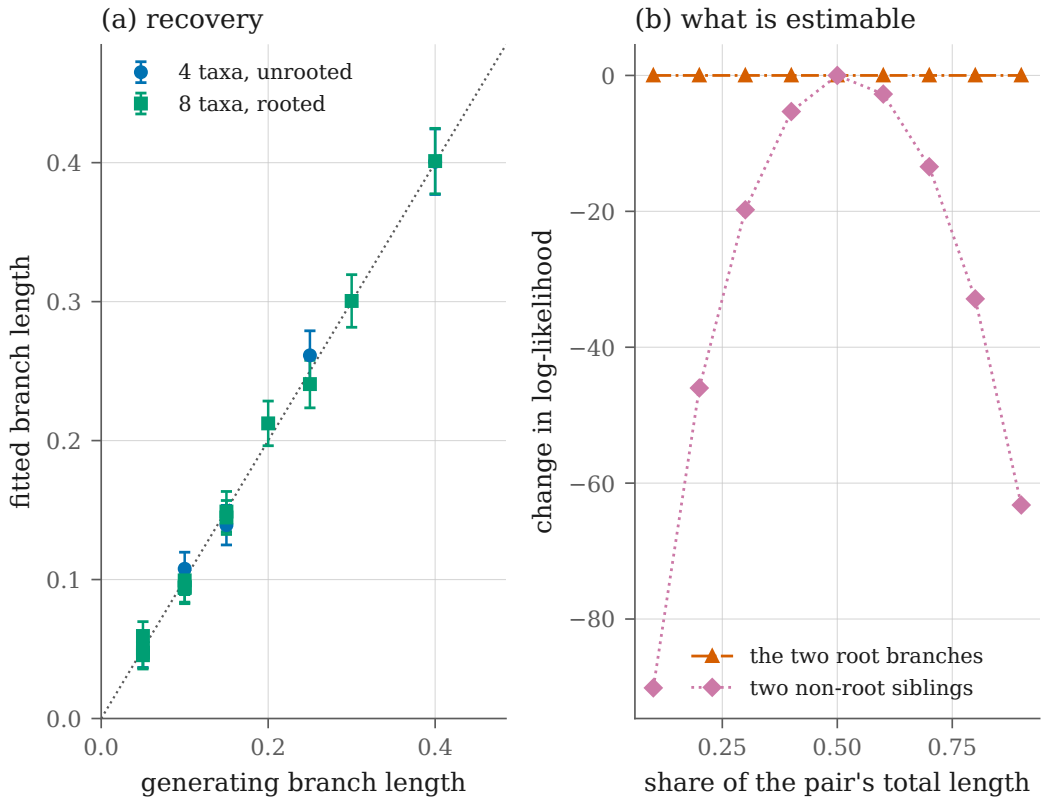

Figure 6: Branch-length fitting by autodiff, using the same model-agnostic optimizer as the Potts and hidden Markov instances. (a) Every estimable branch length against the value that generated the alignment, with 95 percent Wald intervals from the observed information; markers are open where an interval misses. Fixtures: 3-child root, seed 20260903, and 2-child root, seed 20260904, both scored at 5000 sites. (b) Why some branch lengths are not estimable at all. Moving mass between the two branches below a rooted binary tree's root, at fixed sum, changes the log-likelihood not at all: the log-likelihood is bit-identical across a 9 to 1 range. The same move between two non-root siblings changes it by up to 90.1. This is the pulley principle: under a reversible model the likelihood does not depend on where the root sits along the branch it subdivides. Only the pair's sum is estimable, so the objective fits it as one parameter; fitting both would leave the observed information singular and every interval in panel (a) undefined.

structure the data does not contain. Jukes–Cantor is exactly the point of the parameter space where a model this flexible could most easily overfit, so it is the point worth checking.

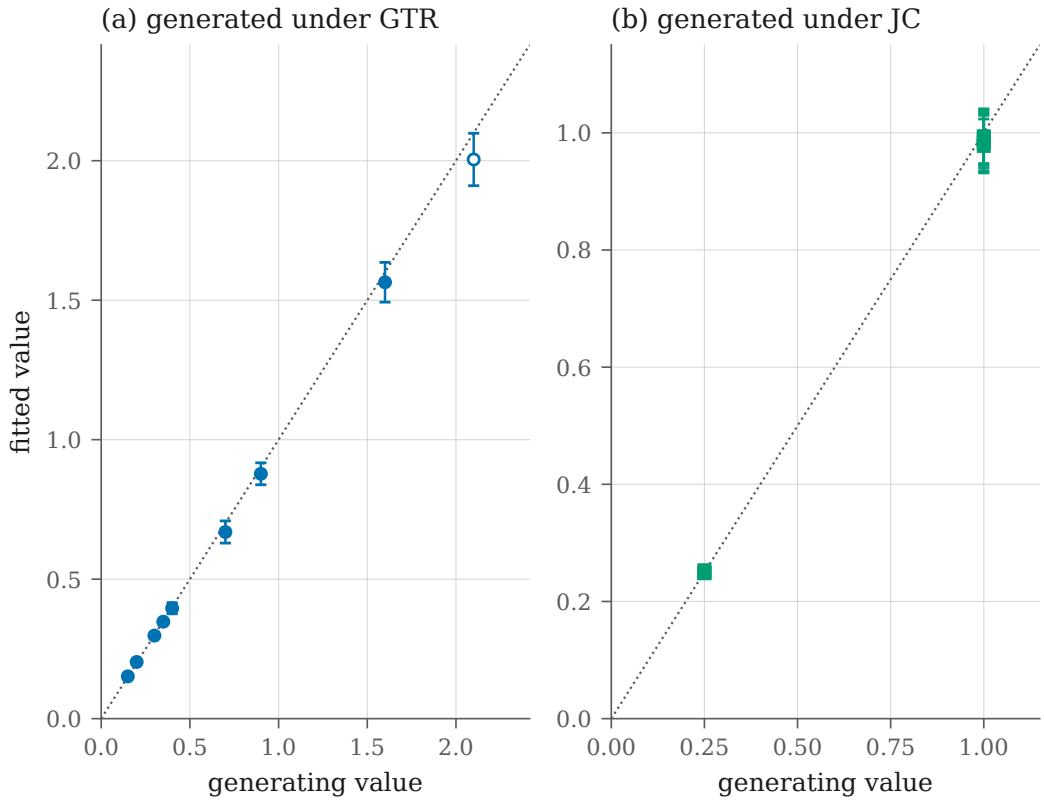

Figure 7: Fitting the general time-reversible model: five free exchangeabilities (the sixth is pinned at 1) and four stationary frequencies, alongside the branch lengths, by the same model-agnostic optimizer used for the Potts and hidden Markov instances. Bars are 95 percent Wald intervals from the observed information; the dotted line is y = x; markers are open where an interval misses. Topology: the 2-child-root fixture, seed 20260904, 20000 sites. (a) Data simulated under an asymmetric truth, 8 of 9 intervals covering. (b) Data simulated under Jukes-Cantor, where every exchangeability is 1 and every frequency 1/4: 9 of 9 covering. The second panel is the half that is easy to forget. A model flexible enough to fit panel (a) is only useful if it also declines to invent structure that is not there, and Jukes-Cantor is exactly the point of the parameter space where the general model could most easily do so. Coverage on a single dataset is a draw and not a rate: the nominal 0.95 is checked over independent replicates by the release-gated regression suite.

Search-trajectory QA figures are not built yet; this section is where their captions go once they are.

10 Sources

The texts this project works from, grouped as `CLAUDE.md` groups them — a routing table keyed on what each informs, rather than a reading list. Where an algorithm we implement appears in one of them, this document follows its formulation and notation and says where it deviates.

10.1 Infrastructure: build, structure, and speed

Software craft. Martin [15] on the structure that keeps a change reviewable; Blandy et al. [3] on ownership, lifetimes, and the FFI boundary the Rust backend crosses.

Systems and hardware. Bryant and O'Hallaron [4] on the memory hierarchy and where CPU time goes; Hwu et al. [11] on GPU kernels, occupancy, and parallel reduction and scan.

10.2 Optimization: discrete and continuous

Algorithms and discrete mathematics. Cormen et al. [6] on data structures, traversal, complexity, and branch and bound; Rosen [24] on the counting and graph theory behind tree space.

Numerical optimization. Nocedal and Wright [19] on line search, trust region, and quasi-Newton methods for the continuous parameters.

Probabilistic inference. MacKay [14] on inference, message passing, and Monte Carlo; Koller and Friedman [12] on factor graphs and the exact-inference machinery that (2) instantiates; Frey [9] on belief propagation read across inference and coding; Ortega [20] on spectral methods for signals indexed by graph vertices.

Statistical physics. Mézard and Montanari [16] on inference over sparse graphs and phase transitions in hard combinatorial problems — tree search being one; Newman and Barkema [17] on sampling, equilibration, and error estimates from correlated samples, which the benchmark harness needs to rank variants rather than noise.

Information geometry. Amari [1] on the statistical manifold, the Fisher metric, and natural gradient.

Learning and reinforcement learning. Goodfellow et al. [10] and Prince [22] on architectures, optimization, and automatic differentiation; Sutton and Barto [25] on MDPs, value and policy methods, and temporally extended actions; Lapan [13] on implementation practice; Raschka [23] on transformer internals and tokenization, for the policy over canonical Newick strings proposed in Section 5.4.

10.3 Application: the science

Phylogenetics. Felsenstein [8] on substitution models, pruning, tree search, and the parsimony and likelihood criteria; Durbin et al. [7] on probabilistic models of sequences; Compeau and Pevzner [5] on the algorithmic view of alignment and phylogeny construction; Pachter and Sturmfels [21] on the algebraic structure of phylogenetic models and their invariants, which offer topology tests that avoid likelihood optimization altogether.

Information and quantum. Blahut [2] on algebraic coding, which is the background for the compressed tree encodings in Section 5.4; Nielsen and Chuang [18] on quantum information and algorithms, background only — nothing here targets quantum hardware.

References

[1] Shun-ichi Amari. *Information Geometry and Its Applications.* Springer, Tokyo, 2016.

[2] Richard E. Blahut. *Algebraic Codes for Data Transmission.* Cambridge University Press, Cambridge, 2003.

[3] Jim Blandy, Jason Orendorff, and Leonora F. S. Tindall. *Programming Rust: Fast, Safe Systems Development*. O'Reilly Media, 2nd edition, 2021.

[4] Randal E. Bryant and David R. O'Hallaron. *Computer Systems: A Programmer's Perspective*. Pearson, 3rd edition, 2015.

[5] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms: An Active Learning Approach*. Active Learning Publishers, 2015.

[6] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, Massachusetts, 4th edition, 2022.

[7] Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, Cambridge, 1998.

[8] Joseph Felsenstein. *Inferring Phylogenies*. Sinauer Associates, Sunderland, Massachusetts, 2004.

[9] Brendan J. Frey. *Graphical Models for Machine Learning and Digital Communication*. MIT Press, Cambridge, Massachusetts, 1998.

[10] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, Massachusetts, 2016.

[11] Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 4th edition, 2022.

[12] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, Cambridge, Massachusetts, 2009.

[13] Maxim Lapan. *Deep Reinforcement Learning Hands-On*. Packt Publishing, 2nd edition, 2020.

[14] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, Cambridge, 2003.

[15] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.

[16] Marc Mézard and Andrea Montanari. *Information, Physics, and Computation*. Oxford University Press, Oxford, 2009.

[17] M. E. J. Newman and G. T. Barkema. *Monte Carlo Methods in Statistical Physics*. Oxford University Press, Oxford, 1999.

[18] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, 10th anniversary edition, 2010.

[19] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.

[20] Antonio Ortega. *Introduction to Graph Signal Processing*. Cambridge University Press, Cambridge, 2022.

[21] Lior Pachter and Bernd Sturmfels, editors. *Algebraic Statistics for Computational Biology*. Cambridge University Press, Cambridge, 2005.

[22] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, Cambridge, Massachusetts, 2023.

[23] Sebastian Raschka. *Build a Large Language Model (From Scratch).* Manning Publications, 2024.

[24] Kenneth H. Rosen. *Discrete Mathematics and Its Applications.* McGraw-Hill, 8th edition, 2019.

[25] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction.* MIT Press, Cambridge, Massachusetts, 2nd edition, 2018.

A Background material

Standard algorithms that are not specific to `phylo`, referenced from the main text rather than re-derived there (see the intended-reader note after the table of contents).

A.1 NNI and SPR

Two rearrangement neighbourhoods define the tree-search graph used in Section 5.4 and throughout [8]:

NNI (nearest-neighbour interchange). Each internal edge of an unrooted binary tree separates four subtrees, which can be rejoined in three ways; two are new topologies. With $n-3$ internal edges, the NNI neighbourhood contains $2(n-3)$ topologies — small, cheap, and local.

SPR (subtree prune and regraft). Detach a subtree and reattach it at another edge. The neighbourhood is quadratic in n, so each step costs more, but the larger reach escapes local optima that trap NNI.

B The substitution model

Character evolution along a branch is a continuous-time Markov chain on $\mathcal{A}$ with rate matrix $\mathbf{Q}$, whose off-diagonal entries $q_{ij} \geq 0$ give the instantaneous rate of substitution from state i to state j and whose rows sum to zero,

$$q_{ii} = -\sum_{j \neq i} q_{ij}. \tag{10}$$

Transition probabilities over a branch of length t follow from the matrix exponential,

$$\mathbf{P}(t) = \exp(\mathbf{Q}t), \qquad P_{ij}(t) = \Pr(X_t = j \mid X_0 = i). \tag{11}$$

$\mathbf{P}(t)$ is a stochastic matrix: $P_{ij}(t) \geq 0$ and each row sums to one. Both properties are invariants the implementation is tested against.

B.1 Reversibility and the root distribution

The models used here are time-reversible with respect to a distribution $\boldsymbol{\pi}$, meaning they satisfy detailed balance,

$$\pi_i q_{ij} = \pi_j q_{ji} \quad \text{for all } i, j. \tag{12}$$

Reversibility has two consequences we rely on. First, $\boldsymbol{\pi}$ is the stationary distribution of the chain, which makes it the natural choice for the distribution of states at the root. Second, the likelihood is invariant to where the root is placed on the tree — Felsenstein's *pulley principle*

[8] — so an unrooted topology suffices for inference, and re-rooting is a test the implementation must pass rather than a modelling decision.

Every reversible rate matrix factorizes in the same way, which is the general time-reversible (GTR) model of Section B.2, eq. (14); the familiar JC69, K80 and HKY85 models are constrained special cases [8]. Branch lengths measure expected substitutions per site, which fixes the scale of $\mathbf{Q}$ through the normalization

$$-\sum_i \pi_i q_{ii} = 1. \tag{13}$$

Without (13), $\mathbf{Q}$ and $\boldsymbol{\tau}$ trade off against each other and neither is identifiable.

B.2 The general time-reversible model

Jukes–Cantor has no free rate parameters, so there is nothing in $\mathbf{Q}$ or $\boldsymbol{\pi}$ to estimate under it. The general reversible model supplies some:

$$\mathbf{Q}_{ij} = s_{ij}\,\pi_j \quad (i \neq j), \qquad \mathbf{Q}_{ii} = -\sum_{j \neq i} \mathbf{Q}_{ij}, \tag{14}$$

with s symmetric — the *exchangeabilities* — and $\boldsymbol{\pi}$ the stationary distribution. Detailed balance, $\pi_i \mathbf{Q}_{ij} = \pi_j \mathbf{Q}_{ji}$, follows immediately from the symmetry of s, so (14) is reversible by construction rather than by assumption, and the eigendecomposition (6) applies exactly.

Two normalizations are gauges rather than conventions, and a third appears once $\boldsymbol{\pi}$ is fitted. Each removes a direction along which the likelihood is exactly flat; a flat direction makes the observed information singular, so without them no parameter of the model has a confidence interval at all. First, $\mathbf{Q}$ and $c\mathbf{Q}$ describe the same process at branch lengths t and t/c, so $\mathbf{Q}$ is scaled to one expected substitution per unit time, $-\sum_i \pi_i \mathbf{Q}_{ii} = 1$; this is what makes a branch length mean "expected substitutions per site". Second, the same freedom reappears in s, since scaling every entry is undone by that normalization, so one entry is held at 1. Third, $\boldsymbol{\pi}$ is optimized through a softmax whose first logit is pinned, since adding a constant to all of them changes nothing.

Setting every s_{ij} equal and $\boldsymbol{\pi}$ uniform must reproduce the Jukes–Cantor rate matrix exactly, which is how the implementation is validated — against a model this document has already pinned against its own closed form, rather than against a restatement of (14).

B.3 The k-state Jukes–Cantor model

The most constrained case is worth stating in closed form, because it gives the simulator an oracle that owes nothing to our likelihood code. Take all exchange rates equal and $\boldsymbol{\pi}$ uniform on k states. Under the normalization (13), the transition probabilities over a branch of length t are

$$P_{ii}(t) = \frac{1}{k} + \frac{k-1}{k}\,e^{-kt/(k-1)}, \qquad P_{ij}(t) = \frac{1}{k} - \frac{1}{k}\,e^{-kt/(k-1)} \quad (i \neq j). \tag{15}$$

At $t = 0$ this gives the identity, and as $t \to \infty$ every entry tends to $1/k$, the stationary distribution. For $k = 4$ it is JC69 [8]. Simulated substitution frequencies are checked against (15) within Monte Carlo error, which tests the simulator independently of the pruning implementation — if both were validated against each other, a shared error would pass unnoticed.